

Ejemplo 2 (XMLHttpRequest + Promesa): Obtener Posts

Objetivo del ejercicio

- Practicar **peticiones AJAX asíncronas** con `XMLHttpRequest` envueltas en **promesas**.
- Mostrar cómo manejar la respuesta con `.then()`, errores con `.catch()` y finalización con `.finally()`.
- Visualizar los datos de la API de manera atractiva usando **tarjetas de Bootstrap**.

Código completo (HTML + Bootstrap + JS)

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ejemplo 2 - XMLHttpRequest con Promesas</title>
  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

  <h1 class="mb-4">Ejemplo 2: Obtener Posts con XMLHttpRequest y
Promesas</h1>
  <p>Haz click en el botón para cargar los primeros 10 posts de la
API pública <code>JSONPlaceholder</code>.</p>

  <button id="btnCargarPosts" class="btn btn-success">Cargar
Posts</button>

  <div id="resultadoPosts" class="mt-3"></div>

  <script>
    const btnCargarPosts =
document.getElementById('btnCargarPosts');
    const resultadoPosts =
document.getElementById('resultadoPosts');

    // Función que devuelve una promesa para obtener posts
    function obtenerPosts() {
      return new Promise((resolve, reject) => {
        const xhr = new XMLHttpRequest();
        xhr.open('GET',
'https://jsonplaceholder.typicode.com/posts', true);

        xhr.onreadystatechange = () => {
          if (xhr.readyState === 4) {
            if (xhr.status === 200) {
              const posts =
JSON.parse(xhr.responseText);
```

```

        resolve(posts); // Resolver la promesa con
los datos
    } else {
        reject(`Error ${xhr.status}:
${xhr.statusText}`);
    }
    };
    xhr.send();
});
}

// Evento click para cargar posts
btnCargarPosts.addEventListener('click', () => {
    resultadoPosts.innerHTML = `<div class="alert alert-
info">Cargando posts...</div>`;

    obtenerPosts()
        .then(posts => {
            let html = '';
            // Mostrar solo los primeros 10 posts
            posts.slice(0, 10).forEach(post => {
                html += `
                <div class="card mb-3">
                    <div class="card-header">Post #${post.id}
- Usuario ${post.userId}</div>
                    <div class="card-body">
                        <h5 class="card-
title">${post.title}</h5>
                        <p class="card-text">${post.body}</p>
                    </div>
                </div>`;
            });
            resultadoPosts.innerHTML = html;
        })
        .catch(error => {
            resultadoPosts.innerHTML = `<div class="alert
alert-danger">${error}</div>`;
        })
        .finally(() => {
            console.log("Petición de posts finalizada");
        });
    });
</script>

</body>
</html>

```

Explicación paso a paso

1. **Función `obtenerPosts()`:**
 - o Crea y devuelve una **promesa** que envuelve la petición AJAX clásica.
 - o `resolve(posts)` se llama cuando la petición es exitosa.
 - o `reject(error)` se llama si hay un fallo (status distinto de 200).
2. **Evento click:**
 - o Llama a `obtenerPosts()` y maneja la respuesta con `.then()`, errores con `.catch()` y finalización con `.finally()`.

3. Procesamiento de datos:

- Convertimos la respuesta JSON a objeto JS con `JSON.parse()`.
- Mostramos los primeros 10 posts en **tarjetas Bootstrap** con título, cuerpo, `postId` y `userId`.

4. Ventaja de usar Promesas:

- La sintaxis es más clara y permite encadenar operaciones futuras, en comparación con callbacks anidados (`callback hell`).

Preguntas de comprensión para alumnos

1. ¿Qué hace `resolve()` y qué hace `reject()` dentro de la promesa?
2. ¿Cómo se decide qué posts mostrar si la API devuelve 100 posts?
3. ¿Por qué se utiliza `slice(0, 10)`?
4. ¿Qué diferencias ves entre este código y usar `fetch()` con promesas?
5. ¿Cómo podrías mejorar la presentación de los posts usando Bootstrap?

Mejoras sugeridas para el alumno

1. Implementar **paginación** para mostrar 5 o 10 posts por página.
2. Añadir un **campo de búsqueda por título** para filtrar posts.
3. Mostrar posts en un **modal** al hacer click en el título para ver más detalles.
4. Manejar el **estado de carga** (por ejemplo, spinner) mientras se realiza la petición.
5. Crear una **función reutilizable** para otras peticiones AJAX con promesas.